让 AI 交付成果，需要一整套系统

从这份 PPT，聊 Agent、 Harness 与持续改进

从一份 PPT 开始

Agent × Harness × Context × Memory × Eval × RSI

官方资料与源码 · 一个贯穿案例 · 三个开放问题

23

页交流素材

约 43 分钟 + 20 分钟讨论

先看一次完整交付，再讨论系统为什么能持续做事。

2026.10.23 · 交流素材

交付一份 PPT，至少要走完六步

同一个目标，需要多次行动、落盘和核验

01 找官方资料 → 02 核事实出处 → 03 整理知识 → 04 设计讲法 → 05 生成 PPT → 06 验证交付

交付标准先写清：可追溯、可编辑、可检查。

本场案例｜不是四套系统的同题实测

固定流程与动态决策可以一起用

LLM 生成内容； Workflow 固定路径； Agent 根据反馈选下一步

01

单次 LLM

输入 → 回答

“解释一下 Harness”

02

Workflow

预定代码路径

“JSON → PPTX → PDF”

03

Agent

观察反馈再选行动

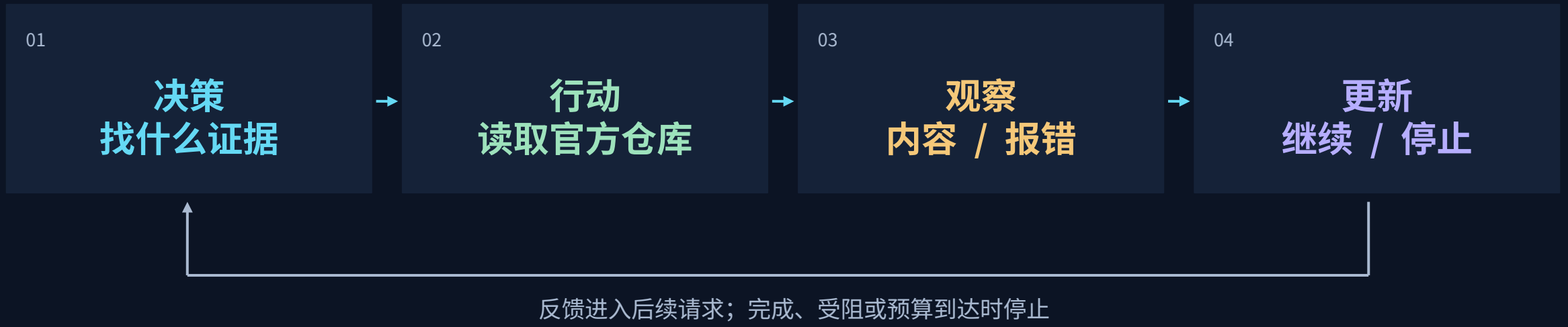
“链接失效 → 换官方来源”

术语采用工作定义；不同项目用法并不完全相同

S22 Agents 定义 S10 Codex loop

Agent Loop 把工具结果带回下一轮

决策 → 行动 → 观察 → 再决策



行动改变环境；观察改变下一次模型输入。

示意架构 | 闭环能适应反馈，但不会自动保证正确

S10 Codex loop S19 会话投影

能调用工具，还不足以可靠交付

失败可能出在信息、执行、连续性和验收四处

01

资料过期

有链接 ≠ 有证据

02

环境缺失

有工具名 ≠ 能执行

03

任务中断

聊天记录 ≠ 恢复点

04

验收遗漏

能构建 ≠ 能交付

先定位失败发生在哪一层，再决定修模型、流程还是环境。

以下是故障注入情景，不是项目性能测量

Harness 承载循环，也管理运行边界

模型提出行动； Harness 决定怎样执行和记录



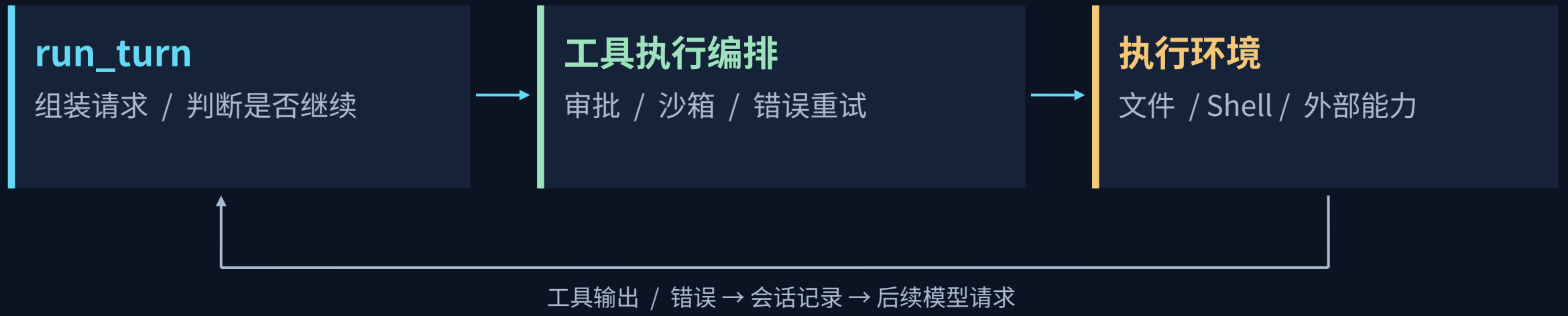
Harness 提高可控性；内容可信度还需要应用自己的验收。

示意架构 | Harness 与 Agent Loop 存在包含和重叠关系

Codex 把循环、工具策略和会话接在一起

从 run_turn 到 ToolOrchestrator，再把反馈写回会话

客户端 / App Server · 线程、轮次、事件



开源的是 Harness 与集成层；不要据此推断模型权重开放。

官方事实 + 简化调用关系｜省略内部模块，不是完整调用图

S10 Codex loop S11 工具策略 S02 Codex harness

Tool 、 MCP 、 Skill 各自补上不同的一环

Tool 提供动作； MCP 连接能力； Skill 组织做事方法

01

Tool · 动作

读取 / 搜索 / 渲染

具体输入和输出

02

MCP · 连接

对接外部工具与资源

协议与服务边界

03

Skill · 方法

怎样研究与验收

说明 + 脚本 + 参考资料

“装了 Skill” 不是质量证书，执行与验收仍要有证据。

本次 ai-tech-talk： 研究规范 + 讲解规则 + 构建脚本

S09 Skills　S02 Codex harness　S21 工具 guard

DSH 用 Cordis 把运行能力组合成插件树

模型适配、工具、会话与 Agent Loop 都有替换接缝

Profile = 有顺序的 bundles + patches

Agent Loop
驱动决策与行动

LLM adapter
模型消息与流

Tools
注册与执行策略

Session
事件日志与投影

CORDIS · services / events / reversible effects

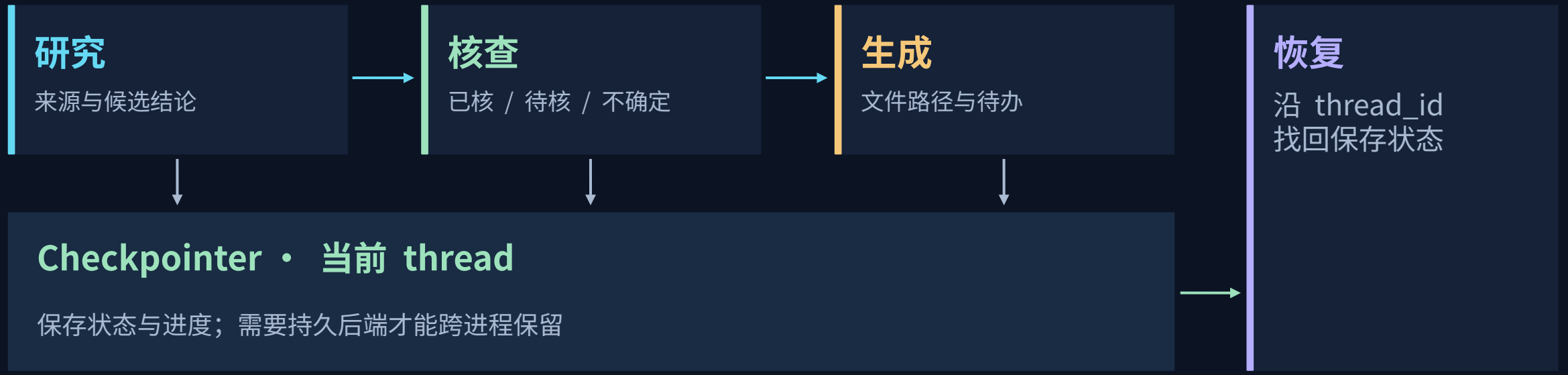
源码观察： AgentLoop extends Service ； 插件化不等于免测试。

官方架构的简化图｜官网仍标注预览版

S03 DSH 官网 S04 DSH 架构 S16 AgentLoop S19 会话投影 S21 工具 guard

LangGraph 把恢复建立在线程状态之上

Graph State → Checkpoint → 同一 thread 继续



Checkpoint 解决“接着做到哪”； Store 解决“别的任务也能用什么”。

示意架构 | 必须选择持久后端；进程内 saver 重启会丢失

OpenHands 把 Agent 行为与执行部署分开

Conversation 选择本地或远程；工具在选定环境运行

Agent + Conversation

决策 / 事件 / 会话状态

工厂按 workspace 类型
选择实现

LocalWorkspace

本地进程
工具在本地环境运行

RemoteWorkspace

远程 Conversation → Agent Server
Agent 与工具在远程环境运行

本地模式与远程模式是部署选择；隔离效果取决于实际配置。

官方 SDK 简化图 | LocalWorkspace 不自动隔离宿主机

四个项目提供的是不同的设计抓手

先问自己缺什么，再选择要借用哪一层

Codex	怎样可控地运行任务	循环、工具政策、线程
DSH	怎样替换和组合能力	Cordis 服务、事件、 profile
LangGraph	怎样保存与恢复进度	Graph / Checkpointer / Store
OpenHands SDK	怎样部署软件 Agent	Conversation / Workspace / Server

比较维度：运行控制、扩展接缝、持久状态、执行环境。

架构归纳｜能力有重叠，可组合，不是四选一或性能榜单

S10 Codex loop　S11 工具策略　S04 DSH 架构　S12 StateGraph　S13 Checkpoint　S17 Conversation

上下文工程决定这一轮模型拿到哪些信息

把任务目标、相关证据和当前进度组装成有效输入

01 选相关证据 → 02 检索原文片段 → 03 组装当前输入 → 04 压缩冗余历史 → 05 保留回查路径

本例：写 checkpoint 页，只带恢复机制的证据与边界。

示意流程｜不是把所有历史都塞进窗口

Context 、 State 、 Memory 是三个观察视角

本轮输入 | 任务记录 | 可复用保留信息

01

Context · 现在看见

这一轮的模型输入

当前源码片段 + 指令

02

State · 当前记录

工作流保存的状态

页数 / 待核事实 / 文件路径

03

Memory · 留作复用

跨时间保留的信息

听众偏好 / 已验证经验

窗口容量、状态持久化、记忆质量，是三个不同问题。

教学区分 | 可重叠； State 里的值也可能记录错

S13 Checkpoint　S18 Store　S14 Agent.step

记忆要经过写入筛选，也要能失效纠错

候选信息 → 验证与定界 → 保存 → 检索 → 更新

State → Checkpoint

当前任务：已核结论 / 待办 / 文件位置

候选
知识 / 经历 / 方法

→

写入筛选
来源 / 范围 / 版本

→

Memory Store
按 namespace 保存

→

检索到 Context
与本轮任务相关

纠错 / 失效 / 复核：更新持久记录，再让后续检索生效

先把“可信地记住与忘记”做好，再决定是否引入向量库。

示意设计｜不是 LangGraph Store 默认提供的完整记忆策略

S18 Store S13 Checkpoint

把错误写成长期事实，会放大后续失误

例：把“内存 saver” 误记为“重启可恢复”

01

错误记忆

有 checkpoint 就不丢

未注明存储后端

02

核对证据

InMemorySaver 在 RAM

重启后不保留

03

修正记录

注明后端与适用范围

旧条目失效并补回归

纠错要同时更新当前回答、持久记录和回归用例。

故障注入示例｜官方文档明确指出内存 saver 的限制

评测先对齐交付标准，再比较改动

用固定任务与独立验收，观察结果、成本和人工介入

事实可追溯	结论 → 官方原文 / commit	人工核支持关系
结构可交付	页数、备注、引用、文件完整	自动 check
视觉可阅读	溢出、重叠、中文、对比度	渲染 + 目检
运行成本	耗时、费用、人工介入	固定环境记录

修改前后用相同任务比较；未见任务留到最后验收。

本案例的评测设计｜阈值是建议，没有虚构实验成绩

构建检查、事实审阅和视觉检查各有盲区

通过一项验收，不能替代另外两项

01

build + check

23 页 / 23 条讲者备注

文件结构与引用检查通过

02

claim → source

22 项来源 / 5 个仓库提交

静态阅读，未部署四项目

03

render + 目检

23 页已渲染并逐页审阅

现场软件兼容性仍未验证

事实出处、构建日志与渲染预览一起交付。

本次交付 | 23 页已构建与渲染；详细检查结果见 QA.md

重试、记忆积累与自修改不是同一种进步

先分清改了任务结果、保留信息，还是 Agent 程序

01

任务内修正

换来源 / 修一页

目标产物变好

02

记忆积累

保存偏好与教训

后续输入可能改善

03

系统自修改

改工具 / Agent 代码

再评测后续改进能力

“自己改了代码”是现象；“持续可靠地变强”需要更强证据。

RSI：递归自我改进；本页为教学分层，不是统一标准

S10 Codex loop S18 Store S20 DGM loop

DGM 用编码评测筛查自修改的 Agent

从候选档案选父代 → 修改代码 → 评测 → 更新档案

01 候选 Agent 档案 → 02 选择父候选 → 03 生成代码改动 → 04 编码任务评测 → 05 更新候选档案

多分支探索，不是每一代只保留一个冠军

经验评测能筛选候选；不足以保证开放环境中的无限递归改进。

研究案例｜不是数学证明，也没有在本次复现性能

S08 DGM 论文　S15 DGM README　S20 DGM loop

可靠改进来自多个闭环彼此连接

Agent 做事； Harness 承载； Context / Memory 供给； Eval 检验



先做可观察、可验收的小闭环，再讨论更大的自主性。

示意架构｜跨项目归纳，非任一官方完整架构

S10 Codex loop　S11 工具策略　S13 Checkpoint　S18 Store　S20 DGM loop

从设计问题进入源码，比从项目名入手更容易

每个问题都有一条可复查的阅读路径

运行控制	Codex: session/turn.rs → tools/orchestrator.rs
插件接缝	DSH: docs/architecture.md → AgentLoop
状态与记忆	LangGraph: BaseCheckpointSaver ↔ BaseStore
执行与演进	OpenHands: Conversation； DGM: DGM_outer.py

阅读重点是责任边界和失败路径，不是记住所有模块名。

完整直链、 commit 、代码摘录与 claims 对照见资料包

用三个真实取舍，交换各自的实践

自主到哪一步？记住哪些东西？怎样证明改得更好？

01

自主性边界

哪一个动作必须由你确认？

速度与控制怎么取舍

02

记忆的边界

哪条信息值得留下或过期？

连续性与错误累积怎么取舍

03

改进的证据

拿什么未见任务验收自修改？

指标提升与真实收益怎么取舍

没有标准答案；具体场景比泛泛站队更有价值。

建议 20 分钟｜每个问题先给一个亲身案例，再谈原则